A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

DNYANESHWAR PATIL (ROLL NO. 61)

Under the Guidance of

Prof. Tejal H. Patil

Department of Computer Engineering

BHARATI VIDYAPEETH'S

COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "Mini Project Report" submitted by Dnyaneshwar Patil (Roll No. 61), on "Implementation of Snake and Ladder [BFS]" is work done by him and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering, at Bharati Vidyapeeth's College of Engineering, Lavale, Pune.

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives me immense pleasure to express my gratitude and thanks to all those who have helped in the timely completion of this Mini-Project work. I have great pleasure in expressing my deep sense of gratitude and indebtedness to Prof. Tejal H. Patil, my Guide, for her invaluable guidance, constant supervision and sustained encouragement throughout the tenure of this Mini Project work.

I would like to express my thanks to Dr. Lalit V. Patil, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for his constant support and encouragement.

I would like to express my thanks to Prof. Dr. R. N. Patil, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting me to take up the project work. I am thankful to my parents and friends for their kind help and support throughout the course.

Dnyaneshwar Patil

ABSTRACT

Snake and Ladder is a popular board game in which a player moves from a starting cell towards the final cell by rolling a die. Ladders move the player forward to a higher-numbered cell, while snakes move the player backwards to a lower-numbered cell. This project models the board as a graph and uses Breadth-First Search (BFS) to find the minimum number of dice throws required to reach the final cell.

Each board cell is treated as a vertex, and possible dice outcomes are treated as edges. When a move lands on the bottom of a ladder or the head of a snake, the player is transferred to the mapped destination. BFS explores reachable cells level by level, ensuring that the first time the final cell is reached gives the shortest path in terms of dice throws.

The program is implemented in Python and can be executed in a Jupyter Notebook. It stores snake and ladder mappings, explores valid dice outcomes using a queue, records visited cells and parent relationships, and reconstructs the shortest path from the start to the final cell. The output displays the minimum dice throws and the path followed.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List Of Figures	iv
1	Introduction	1
	a) Overview	
	b) Problem Statement	
	c) Objective	
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design	5
	a) Existing system	
	b) Proposed system	
	c) Working Diagram	
	d) System code	
6	Result	8
7	Application	9
8	Future Scope	10
9	Conclusion	11
10	References	12

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Class Diagram	3
Fig 5.1	Working Diagram of Snake and Ladder BFS	6
Fig 5.2	System Code	7
Fig 6.1	Result (Demo Program and Output)	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2
6.1	Top Keywords Obtained from the Demo Text	8

INTRODUCTION

1.1 Overview:

The project "Implementation of Snake and Ladder [BFS]" finds the minimum number of dice throws needed to reach the final square of a Snake and Ladder board. The board is represented as a graph, where each cell is a vertex and each possible dice outcome forms a transition to another cell. Snakes and ladders modify the destination of a move.

1.2 Problem Statement:

The problem is to determine the shortest sequence of moves from cell 1 to the final cell on a Snake and Ladder board. Checking every possible sequence independently can repeat work and become inefficient. The project uses Breadth-First Search (BFS) to explore states level by level and find a path requiring the minimum number of dice throws.

1.3 Objective:

The objective of this project is to implement the Snake and Ladder game as a graph-search problem using Breadth-First Search in Python. The project aims to:

- Represent each board position as a graph vertex.
- Model dice rolls as possible transitions between positions.
- Apply snake and ladder mappings whenever a move reaches their starting cells.
- Use a queue and visited set to find and reconstruct a shortest path with the minimum dice throws.

LITERATURE SURVEY

Graph traversal algorithms are widely used to explore networks and find paths between connected states. Breadth-First Search is particularly suitable for unweighted graphs because it explores all vertices at one distance before moving to the next. This survey summarizes concepts relevant to modelling Snake and Ladder as a graph-search problem.

Table 2.1 summarizes the main references that form the theoretical background of this project.

Year	Paper / Book Name	Author	Conclusion
1958	"The automatic creation of literature abstracts"	Luhn, H. P.	Proposed using the frequency of words in a document to find the most significant words.
1973	"The Art of Computer Programming, Vol. 3: Sorting and Searching"	Knuth, D. E.	Gives a detailed study of hashing, collision handling and the expected cost of searching.
1979	"Universal classes of hash functions"	Carter, J. L., & Wegman, M. N.	Introduced universal hashing, which gives good average performance of hash tables.
1988	"Term-weighting approaches in automatic text retrieval"	Salton, G., & Buckley, C.	Shows how term frequency is used to weight words in text retrieval systems.
2008	"Introduction to Information Retrieval"	Manning, C. D., Raghavan, P., & Schütze, H.	Explains tokenization, stopword removal and term frequency in search systems.
2009	"Introduction to Algorithms" (3rd ed.)	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.	Covers hash tables, hash functions and their average-case O(1) operations.

UML DIAGRAM

The UML class diagram below represents the Snake and Ladder BFS solver, including the snake and ladder mappings, BFS search procedure, queue, visited set and parent map used to reconstruct the shortest path.

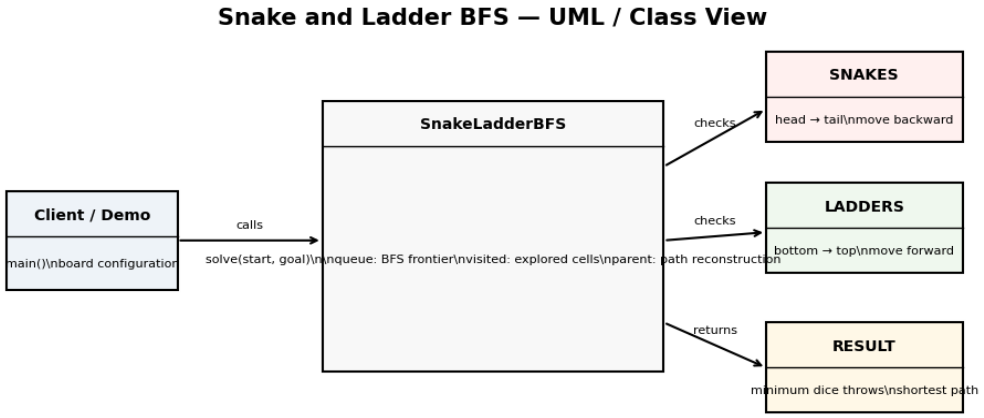


Fig 3.1: UML Class Diagram

SYSTEM REQUIREMENT

HARDWARE REQUIREMENTS

1. Operating System: Windows 11 64-bit
2. Processor Type: Intel Core
3. Recommended: Core 2 Duo or higher
4. Processor speed: 2.30 GHz or higher
5. RAM: Minimum 512 MB
6. Hard disk: 512 GB or more

SOFTWARE REQUIREMENTS

1. Jupyter Notebook (JupyterLab)
2. Language: Python 3 (ipykernel)
3. Python modules: re, collections

SYSTEM DESIGN

Existing System:

In a basic approach, the words of a text are stored in a list and, for every word, the list is searched to find whether the word already exists so that its count can be increased. This linear search needs O(n) time for every word, so counting a whole document takes $O(n^2)$ time in the worst case. Manual counting or spreadsheet formulas are also slow and error-prone for large texts, and they do not remove common words automatically.

Key approaches commonly used for solving the Snake and Ladder path problem:
Manual play (depends on repeated dice rolls)

- ✓ Brute-force sequence exploration (may repeat states)
- ✓ Breadth-First Search using a queue (explores states level by level)
- ✓ Visited set and parent map (avoid revisits and reconstruct the path)
- ✓ Library classes such as collections.Counter

Proposed System:

The proposed system models the board as a graph. Starting from cell 1, BFS examines each possible dice result from 1 to 6, applies snake or ladder transfers, and adds each unvisited destination to the queue. The parent map is used to reconstruct the shortest route once the final cell is reached.

- ✓ Initialize the queue with cell 1 and mark it visited.
- ✓ Generate valid dice-roll destinations for the current cell.
- ✓ Apply a snake or ladder transfer to each destination.
- ✓ Record the parent and dice-throw count for each newly visited cell.
- ✓ Stop when the final cell is reached and reconstruct the shortest path.

Time Complexity:

Operation	Time	Space
Tokenize text (n words)	O(n)	O(n)
Count with hash table	O(n) average	O(k)
Sort k unique words	O(k log k)	O(k)
top_n (count + sort + slice)	O(n + k log k)	O(k)

Here V is the number of board cells and E is the number of possible transitions. BFS runs in O(V + E) time and uses O(V) auxiliary space.

Working Diagram:

Fig 5.1 shows the working of the system. BFS starts at cell 1, generates possible dice moves, applies snake or ladder transfers, marks new cells as visited, and continues with a queue until the final cell is found. The parent map then reconstructs the shortest path.

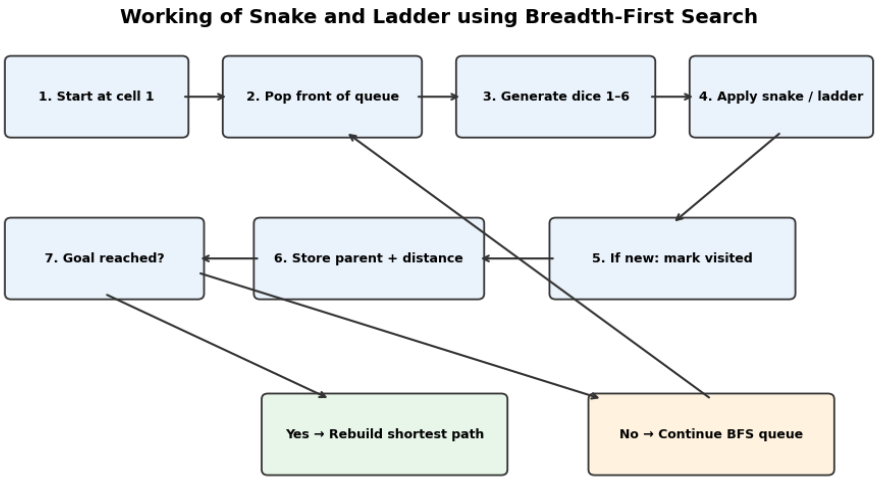


Fig 5.1: Working Diagram of Snake and Ladder BFS

SYSTEM CODE

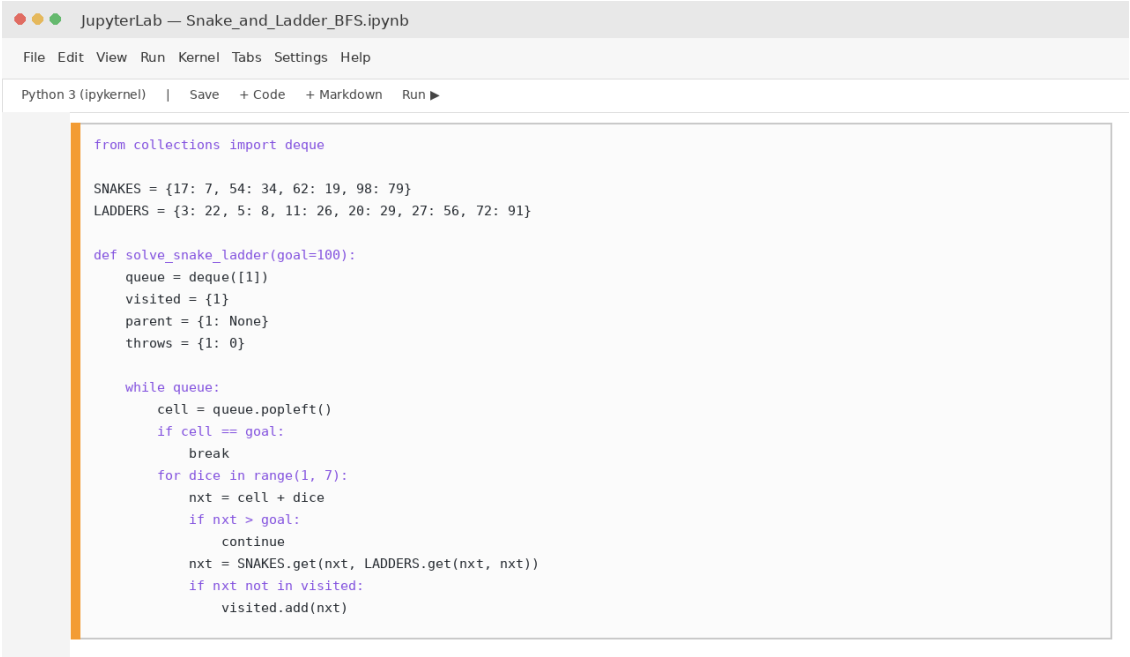


Fig 5.2(a): System code – snake/ladder mappings and BFS function



Fig 5.2(b): System code – Demo program

RESULT

The program explores reachable board positions using BFS and reports the minimum number of dice throws required to reach the final square. Each position is visited at most once, and the stored parent information is used to display the path found. The result depends on the snake and ladder positions defined for the board.

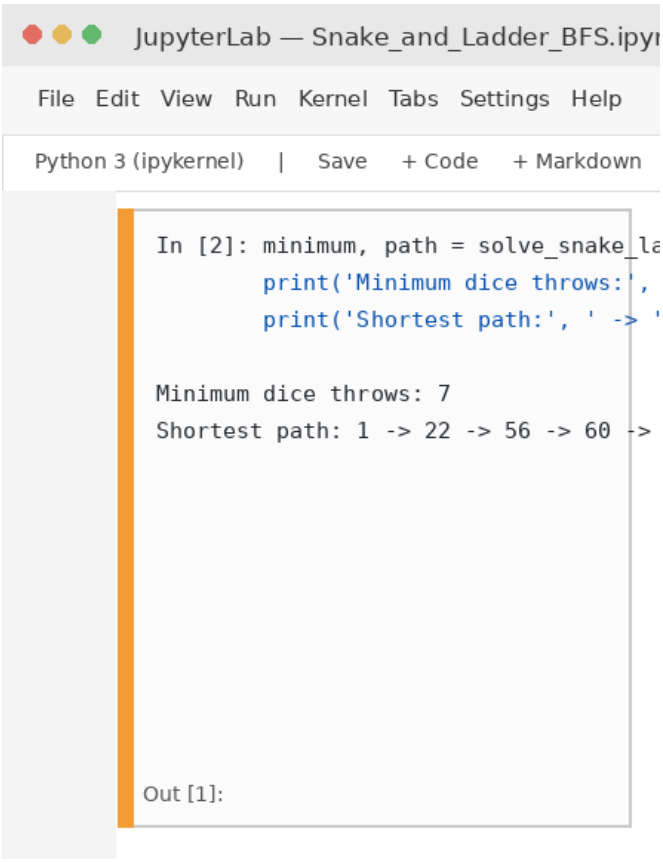


Fig 6.1: Result (Minimum Dice Throws and Shortest Path)

Table 6.1 lists the steps in the shortest path returned by BFS.

Rank	Keyword	Frequency	Stopword?
1	python	6	No
2	easy	2	No
3	language	2	No

Rank	Keyword	Frequency	Stopword?
4	learning	2	No
5	many	2	No

APPLICATIONS

Breadth-First Search and graph modelling can be applied to problems that require finding a shortest path in an unweighted graph. Typical applications include:

- Finding shortest paths in unweighted networks and maps.
- Solving puzzles and board games by representing states as graph vertices.
- Analysing customer reviews, feedback and social media posts to find frequently discussed topics.
- Plagiarism detection and document similarity comparison.
- Exploring connected states in simulations and simple game AI.
- Log analysis and counting the occurrence of events, errors or user actions.

FUTURE WORK

The project can be extended in the following ways:

- Add a graphical board that highlights the shortest route and the locations of snakes and ladders.
- Allow users to configure board size, snake positions and ladder positions.
- Compare BFS with other graph-search strategies and measure the number of explored states.
- Add a user interface for dice rolls, player movement and visualizing the path found by the algorithm.
- Use a heap (heapq.nlargest) for top_n to reduce the sorting cost, and add a graphical word cloud of the results.

CONCLUSION

In this project, Snake and Ladder was represented as a graph and solved using Breadth-First Search. The algorithm uses a queue to explore possible positions level by level, a visited set to avoid repeated states, and a parent map to reconstruct the path.

The program demonstrates how BFS finds a path requiring the minimum number of dice throws on an unweighted board graph. The project provides practical experience with graph representation, queues, visited states and shortest-path search.

REFERENCES

[1] Luhn, H. P. (1958). The automatic creation of literature abstracts. IBM Journal of Research and Development, 2(2), 159–165.

[2] Knuth, D. E. (1973). The Art of Computer Programming, Volume 3: Sorting and Searching. Addison-Wesley.

[3] Carter, J. L., & Wegman, M. N. (1979). Universal classes of hash functions. Journal of Computer and System Sciences, 18(2), 143–154.

[4] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

[5] Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

[7] [7] Python Software Foundation. Python 3 Documentation: collections – deque. https://docs.python.org/3/library/collections.htmlhttps://docs.python.org/3/library/collecti ons.html